

Phụ lục D – Autodiff (Tự động tính đạo hàm)

Notebook này chứa các triển khai thử nghiệm về các kỹ thuật autodiff khác nhau để giải thích cách chúng hoạt động.

 [Open in Colab](#)  [Open in Kaggle](#)

Cài đặt

▼ Giới thiệu

Giả sử chúng ta muốn tính đạo hàm của hàm số $f(x, y) = x^2y + y + 2$ theo các tham số x và y :

```
def f(x,y):  
    return x*x*y + y + 2
```

Một cách tiếp cận là giải bằng phương pháp giải tích:

$$\frac{\partial f}{\partial x} = 2xy$$

$$\frac{\partial f}{\partial y} = x^2 + 1$$

```
def df(x,y):  
    return 2*x*y, x*x + 1
```

Ví dụ, với $\frac{\partial f}{\partial x}(3, 4) = 24$ và $\frac{\partial f}{\partial y}(3, 4) = 10$.

```
df(3, 4)
```

```
(24, 10)
```

Tuyệt vời! Chúng ta cũng có thể tìm các phương trình cho đạo hàm cấp hai (còn được gọi là Hessians):

$$\frac{\partial^2 f}{\partial x \partial x} = \frac{\partial(2xy)}{\partial x} = 2y$$

$$\frac{\partial^2 f}{\partial x \partial y} = \frac{\partial(2xy)}{\partial y} = 2x$$

$$\frac{\partial^2 f}{\partial y \partial x} = \frac{\partial(x^2 + 1)}{\partial x} = 2x$$

$$\frac{\partial^2 f}{\partial y \partial y} = \frac{\partial(x^2 + 1)}{\partial y} = 0$$

Tại $x=3$ và $y=4$, các giá trị Hessian này lần lượt là 8, 6, 6, 0. Hãy sử dụng các phương trình trên để tính toán chúng:

```
def d2f(x, y):  
    return [2*y, 2*x], [2*x, 0]
```

```
d2f(3, 4)
```

```
([8, 6], [6, 0])
```

Rất tốt, nhưng điều này đòi hỏi công sức tính toán toán học. Trong trường hợp này thì không quá khó, nhưng đối với một mạng nơ-ron sâu (deep neural network), việc tính đạo hàm theo cách này gần như là không thể. Vì vậy, hãy xem xét các cách khác nhau để tự động hóa việc này!

✓ Tính đạo hàm bằng phương pháp số (Numeric differentiation)

Ở đây, chúng ta tính toán một giá trị xấp xỉ của đạo hàm bằng phương trình: $\frac{\partial f}{\partial x} = \lim_{\epsilon \rightarrow 0} \frac{f(x + \epsilon, y) - f(x, y)}{\epsilon}$ (và có một định nghĩa tương tự cho $\frac{\partial f}{\partial y}$).

```
def gradients(func, vars_list, eps=0.0001):
    partial_derivatives = []
    base_func_eval = func(*vars_list)
    for idx in range(len(vars_list)):
        tweaked_vars = vars_list[:]
        tweaked_vars[idx] += eps
        tweaked_func_eval = func(*tweaked_vars)
        derivative = (tweaked_func_eval - base_func_eval) / eps
        partial_derivatives.append(derivative)
    return partial_derivatives
```

```
def df(x, y):
    return gradients(f, [x, y])
```

```
df(3, 4)
```

```
[24.0004000000048216, 10.000000000047748]
```

Kết quả khá tốt!

Tin tốt là việc tính toán các Hessian cũng khá dễ dàng. Đầu tiên, hãy tạo các hàm tính toán đạo hàm riêng cấp một (còn được gọi là Jacobians):

```
def dfdx(x, y):
    return gradients(f, [x, y])[0]
```

```
def dfdy(x, y):
    return gradients(f, [x, y])[1]
```

```
dfdx(3., 4.), dfdy(3., 4.)
```

```
(24.0004000000048216, 10.000000000047748)
```

Bây giờ chúng ta chỉ cần áp dụng hàm `gradients()` cho các hàm này:

```
def d2f(x, y):
    return [gradients(dfdx, [x, y]), gradients(dfdy, [x, y])]
```

```
d2f(3, 4)
```

```
[[7.999999951380232, 6.000099261882497],
 [6.000099261882497, -1.4210854715202004e-06]]
```

Mọi thứ hoạt động ổn, nhưng kết quả chỉ là xấp xỉ, và việc tính đạo hàm của một hàm số với n biến yêu cầu gọi hàm đó n lần. Trong các mạng nơ-ron sâu, thường có hàng ngàn tham số cần tinh chỉnh bằng gradient descent (yêu cầu tính đạo hàm của hàm mất mát với từng tham số này), vì vậy cách tiếp cận này sẽ quá chậm.

✓ Triển khai một Đồ thị Tính toán (Computation Graph) thử nghiệm

Thay vì cách tiếp cận số học này, hãy triển khai một số kỹ thuật autodiff ký hiệu (symbolic autodiff). Để làm được điều này, chúng ta cần định nghĩa các lớp (classes) để đại diện cho các hằng số (constants), biến (variables) và các phép toán (operations).

```
class Const(object):
    def __init__(self, value):
        self.value = value
    def evaluate(self):
```

```

        return self.value
    def __str__(self):
        return str(self.value)

class Var(object):
    def __init__(self, name, init_value=0):
        self.value = init_value
        self.name = name
    def evaluate(self):
        return self.value
    def __str__(self):
        return self.name

class BinaryOperator(object):
    def __init__(self, a, b):
        self.a = a
        self.b = b

class Add(BinaryOperator):
    def evaluate(self):
        return self.a.evaluate() + self.b.evaluate()
    def __str__(self):
        return "{} + {}".format(self.a, self.b)

class Mul(BinaryOperator):
    def evaluate(self):
        return self.a.evaluate() * self.b.evaluate()
    def __str__(self):
        return "{} * {}".format(self.a, self.b)

```

Tốt rồi, bây giờ chúng ta có thể xây dựng một đồ thị tính toán để đại diện cho hàm f :

```

x = Var("x")
y = Var("y")
f = Add(Mul(Mul(x, x), y), Add(y, Const(2))) # f(x,y) = x²y + y + 2

```

Và chúng ta có thể chạy đồ thị này để tính f tại bất kỳ điểm nào, ví dụ $f(3, 4)$.

```

x.value = 3
y.value = 4
f.evaluate()

```

42

Hoàn hảo, nó đã tìm ra kết quả cuối cùng.

▼ Tính toán đạo hàm

Các phương pháp autodiff mà chúng tôi trình bày dưới đây đều dựa trên *quy tắc chuỗi* (*chain rule*).

Giả sử chúng ta có hai hàm u và v , và chúng ta áp dụng chúng tuần tự cho đầu vào x , nhận được kết quả z . Như vậy chúng ta có $z = v(u(x))$, có thể viết lại là $z = v(s)$ và $s = u(x)$. Bây giờ chúng ta có thể áp dụng quy tắc chuỗi để tính đạo hàm riêng của đầu ra z theo đầu vào x :

$$\frac{\partial z}{\partial x} = \frac{\partial s}{\partial x} \cdot \frac{\partial z}{\partial s}$$

Nếu z là kết quả của một chuỗi các hàm có các đầu ra trung gian $s_1, s_2, \dots, s_n$, quy tắc chuỗi vẫn được áp dụng:

$$\frac{\partial z}{\partial x} = \frac{\partial s_1}{\partial x} \cdot \frac{\partial s_2}{\partial s_1} \cdot \frac{\partial s_3}{\partial s_2} \cdot \dots \cdot \frac{\partial s_{n-1}}{\partial s_{n-2}} \cdot \frac{\partial s_n}{\partial s_{n-1}} \cdot \frac{\partial z}{\partial s_n}$$

Trong **forward mode autodiff** (tự động tính đạo hàm thuận), thuật toán tính toán các số hạng này theo chiều "xuôi" (tức là cùng thứ tự với các phép tính cần thiết để tính đầu ra z), từ trái sang phải: đầu tiên là $\frac{\partial s_1}{\partial x}$, sau đó là $\frac{\partial s_2}{\partial s_1}$, v.v. Trong **reverse mode autodiff** (tự động

tính đạo hàm nghịch), thuật toán tính toán các số hạng này theo chiều "ngược lại", từ phải sang trái: đầu tiên là $\frac{\partial z}{\partial s_n}$, sau đó là $\frac{\partial s_n}{\partial s_{n-1}}$, và cứ tiếp tục như vậy.

Ví dụ: giả sử bạn muốn tính đạo hàm của hàm số $z(x) = \sin(x^2)$ tại $x=3$ bằng forward mode autodiff. Thuật toán trước tiên sẽ tính đạo hàm riêng $\frac{\partial s_1}{\partial x} = \frac{\partial x^2}{\partial x} = 2x = 6$. Tiếp theo, nó sẽ tính $\frac{\partial z}{\partial x} = \frac{\partial s_1}{\partial x} \cdot \frac{\partial z}{\partial s_1} = 6 \cdot \frac{\partial \sin(s_1)}{\partial s_1} = 6 \cdot \cos(s_1) = 6 \cdot \cos(3^2) \approx -5.46$.

Hãy kiểm tra kết quả này bằng hàm `gradients()` đã định nghĩa trước đó:

```
from math import sin

def z(x):
    return sin(x**2)

gradients(z, [3])

[-5.46761419430053]
```

Kết quả có vẻ ổn. Bây giờ hãy làm tương tự bằng reverse mode autodiff. Lần này thuật toán sẽ bắt đầu từ phía bên phải nên nó sẽ tính

$$\frac{\partial z}{\partial s_1} = \frac{\partial \sin(s_1)}{\partial s_1} = \cos(s_1) = \cos(3^2) \approx -0.91. \text{ Tiếp theo, nó sẽ tính}$$
$$\frac{\partial z}{\partial x} = \frac{\partial s_1}{\partial x} \cdot \frac{\partial z}{\partial s_1} \approx \frac{\partial s_1}{\partial x} \cdot -0.91 = \frac{\partial x^2}{\partial x} \cdot -0.91 = 2x \cdot -0.91 = 6 \cdot -0.91 = -5.46.$$

Tất nhiên cả hai cách tiếp cận đều cho cùng một kết quả (ngoại trừ sai số làm tròn), và với một đầu vào và một đầu ra duy nhất, chúng có cùng khối lượng tính toán. Nhưng khi có nhiều đầu vào hoặc nhiều đầu ra, hiệu suất của chúng có thể rất khác nhau. Thật vậy, nếu có nhiều đầu vào, các số hạng ngoài cùng bên phải sẽ cần thiết để tính đạo hàm riêng cho từng đầu vào, vì vậy tốt nhất nên tính các số hạng bên phải trước. Điều đó có nghĩa là sử dụng reverse-mode autodiff. Bằng cách này, các số hạng bên phải có thể được tính chỉ một lần và dùng cho tất cả các đạo hàm riêng. Ngược lại, nếu có nhiều đầu ra, forward-mode thường được ưu tiên hơn vì các số hạng bên trái có thể được tính một lần duy nhất để tính đạo hàm của các đầu ra khác nhau. Trong Deep Learning, thường có hàng ngàn tham số mô hình, nghĩa là có rất nhiều đầu vào, nhưng rất ít đầu ra. Thực tế, thường chỉ có một đầu ra duy nhất trong quá trình huấn luyện: hàm mất mát (loss). Đây là lý do tại sao reverse mode autodiff được sử dụng trong TensorFlow và tất cả các thư viện Deep Learning chính.

Có một sự phức tạp bổ sung trong reverse mode autodiff: giá trị của s_i thường được yêu cầu khi tính $\frac{\partial s_{i+1}}{\partial s_i}$, và việc tính s_i yêu cầu tính s_{i-1} trước đó, v.v. Vì vậy, về cơ bản, cần một lượt đi thuận (forward pass) qua mạng để tính $s_1, s_2, \dots, s_n$, sau đó thuật toán mới có thể tính các đạo hàm riêng từ phải sang trái. Việc lưu trữ tất cả các giá trị trung gian s_i trong RAM đôi khi là một vấn đề, đặc biệt là khi xử lý hình ảnh và khi sử dụng GPU thường có RAM hạn chế. Để hạn chế vấn đề này, người ta có thể giảm số lượng lớp trong mạng nơ-ron hoặc cấu hình TensorFlow để hoán đổi các giá trị này từ RAM GPU sang RAM CPU. Một cách tiếp cận khác là chỉ lưu bộ nhớ đệm cho mỗi giá trị trung gian khác, ví dụ $s_1, s_3, s_5, \dots$. Điều này có nghĩa là khi thuật toán tính đạo hàm riêng, nếu thiếu giá trị trung gian s_i , nó sẽ cần tính lại dựa trên giá trị trung gian s_{i-1} trước đó. Đây là sự đánh đổi giữa CPU và RAM.

✓ Forward mode autodiff (Đạo hàm thuận)

```
Const.gradient = lambda self, var: Const(0)
Var.gradient = lambda self, var: Const(1) if self is var else Const(0)
Add.gradient = lambda self, var: Add(self.a.gradient(var), self.b.gradient(var))
Mul.gradient = lambda self, var: Add(Mul(self.a, self.b.gradient(var)), Mul(self.a.gradient(var), self.b))

x = Var(name="x", init_value=3.)
y = Var(name="y", init_value=4.)
f = Add(Mul(Mul(x, x), y), Add(y, Const(2))) # f(x,y) = x^2y + y + 2

dfdx = f.gradient(x) # 2xy
dfdy = f.gradient(y) # x^2 + 1
```

```
dfdx.evaluate(), dfdy.evaluate()
```

```
(24.0, 10.0)
```

Vì đầu ra của phương thức `gradient()` hoàn toàn mang tính ký hiệu, chúng ta không bị giới hạn ở đạo hàm cấp một, chúng ta cũng có thể tính đạo hàm cấp hai, và cứ tiếp tục như vậy:

```
d2fdxdx = dfdx.gradient(x) # 2y
d2fdxdy = dfdx.gradient(y) # 2x
d2fdydx = dfdy.gradient(x) # 2x
d2fdydy = dfdy.gradient(y) # 0
```

```
[[d2fdxdx.evaluate(), d2fdxdy.evaluate()],
 [d2fdydx.evaluate(), d2fdydy.evaluate()]]
```

```
[[8.0, 6.0], [6.0, 0.0]]
```

Lưu ý rằng kết quả bây giờ là chính xác, không phải là xấp xỉ (tất nhiên là trong giới hạn độ chính xác số thực dấu phẩy động của máy tính).

✓ Forward mode autodiff sử dụng số dual (dual numbers)

Một cách hay để áp dụng forward mode autodiff là sử dụng [số dual](#). Tóm lại, một số dual z có dạng $z = a + b\epsilon$, trong đó a và b là các số thực, và ϵ là một số cực nhỏ (infinitesimal), dương nhưng nhỏ hơn tất cả các số thực, và sao cho $\epsilon^2 = 0$. Người ta có thể chứng minh được rằng $f(x + \epsilon) = f(x) + \frac{\partial f}{\partial x}\epsilon$, vì vậy đơn giản bằng cách tính $f(x + \epsilon)$, chúng ta có được cả giá trị của $f(x)$ và đạo hàm riêng của f theo x .

Số dual có các quy tắc số học riêng, nhìn chung khá tự nhiên. Ví dụ:

Phép cộng

$$(a_1 + b_1\epsilon) + (a_2 + b_2\epsilon) = (a_1 + a_2) + (b_1 + b_2)\epsilon$$

Phép nhân

$$(a_1 + b_1\epsilon) \times (a_2 + b_2\epsilon) = (a_1 a_2) + (a_1 b_2 + a_2 b_1)\epsilon$$

Phép chia

$$\frac{a_1 + b_1\epsilon}{a_2 + b_2\epsilon} = \frac{a_1}{a_2} + \frac{a_1 b_2 - b_1 a_2}{a_2^2}\epsilon$$

Lũy thừa

$$(a + b\epsilon)^n = a^n + (na^{n-1}b)\epsilon$$

Hãy tạo một lớp (class) để đại diện cho các số dual và triển khai một vài phép toán (cộng và nhân). Bạn có thể thử thêm các phép toán khác nếu muốn.

```
class DualNumber(object):
    def __init__(self, value=0.0, eps=0.0):
        self.value = value
        self.eps = eps
    def __add__(self, b):
        return DualNumber(self.value + self.to_dual(b).value,
                           self.eps + self.to_dual(b).eps)
    def __radd__(self, a):
        return self.to_dual(a).__add__(self)
    def __mul__(self, b):
        return DualNumber(self.value * self.to_dual(b).value,
                           self.eps * self.to_dual(b).value + self.value * self.to_dual(b).eps)
    def __rmul__(self, a):
        return self.to_dual(a).__mul__(self)
    def __str__(self):
        if self.eps:
            return "{:.1f} + {:.1f}\epsilon".format(self.value, self.eps)
        else:
            return "{:.1f}".format(self.value)
    def __repr__(self):
        return str(self)
    @classmethod
    def to_dual(cls, n):
```

```

if hasattr(n, "value"):
    return n
else:
    return cls(n)

```

$$3 + (3 + 4\epsilon) = 6 + 4\epsilon$$

```
3 + DualNumber(3, 4)
```

```
6.0 + 4.0ε
```

$$(3 + 4\epsilon) \times (5 + 7\epsilon) = 3 \times 5 + 3 \times 7\epsilon + 4\epsilon \times 5 + 4\epsilon \times 7\epsilon = 15 + 21\epsilon + 20\epsilon + 28\epsilon^2 = 15 + 41\epsilon$$

```
DualNumber(3, 4) * DualNumber(5, 7)
```

```
15.0 + 41.0ε
```

Bây giờ hãy xem liệu các số dual có hoạt động với khung công việc (framework) thử nghiệm của chúng ta không:

```

x.value = DualNumber(3.0)
y.value = DualNumber(4.0)

```

```
f.evaluate()
```

```
42.0
```

Đúng vậy, nó hoạt động tốt. Bây giờ hãy sử dụng điều này để tính các đạo hàm riêng của f theo x và y tại $x=3$ và $y=4$:

```

x.value = DualNumber(3.0, 1.0) # 3 + ε
y.value = DualNumber(4.0)      # 4

```

```
dfdx = f.evaluate().eps
```

```

x.value = DualNumber(3.0)      # 3
y.value = DualNumber(4.0, 1.0) # 4 + ε

```

```
dfdy = f.evaluate().eps
```

```
dfdx
```

```
24.0
```

```
dfdy
```

```
10.0
```

Tuyệt vời! Tuy nhiên, trong triển khai này, chúng ta bị giới hạn ở các đạo hàm cấp một. Bây giờ hãy cùng tìm hiểu về reverse mode.

▼ Reverse mode autodiff (Đạo hàm nghịch)

Hãy viết lại framework thử nghiệm của chúng ta để thêm reverse mode autodiff:

```

class Const(object):
    def __init__(self, value):
        self.value = value
    def evaluate(self):
        return self.value
    def backpropagate(self, gradient):
        pass
    def __str__(self):
        return str(self.value)

class Var(object):
    def __init__(self, name, init_value=0):
        self.value = init_value
        self.name = name
        self.gradient = 0

```

```

def evaluate(self):
    return self.value
def backpropagate(self, gradient):
    self.gradient += gradient
def __str__(self):
    return self.name

class BinaryOperator(object):
    def __init__(self, a, b):
        self.a = a
        self.b = b

class Add(BinaryOperator):
    def evaluate(self):
        self.value = self.a.evaluate() + self.b.evaluate()
        return self.value
    def backpropagate(self, gradient):
        self.a.backpropagate(gradient)
        self.b.backpropagate(gradient)
    def __str__(self):
        return "{} + {}".format(self.a, self.b)

class Mul(BinaryOperator):
    def evaluate(self):
        self.value = self.a.evaluate() * self.b.evaluate()
        return self.value
    def backpropagate(self, gradient):
        self.a.backpropagate(gradient * self.b.value)
        self.b.backpropagate(gradient * self.a.value)
    def __str__(self):
        return "({}) * {}".format(self.a, self.b)

```

```

x = Var("x", init_value=3)
y = Var("y", init_value=4)
f = Add(Mul(Mul(x, x), y), Add(y, Const(2))) # f(x,y) = x²y + y + 2

```

```

result = f.evaluate()
f.backpropagate(1.0)

```

```
print(f)
```

```
((x) * (x)) * (y) + y + 2
```

```
result
```

```
42
```

```
x.gradient
```

```
24.0
```

```
y.gradient
```

```
10.0
```

Một lần nữa, trong triển khai này, các đầu ra chỉ là các con số, không phải các biểu thức ký hiệu, vì vậy chúng ta bị giới hạn ở các đạo hàm cấp một. Tuy nhiên, chúng ta có thể làm cho các phương thức `backpropagate()` trả về các biểu thức ký hiệu thay vì các giá trị (ví dụ: trả về `Add(2,3)` thay vì 5). Điều này sẽ cho phép tính toán các gradient cấp hai (và xa hơn nữa). Đây là những gì TensorFlow thực hiện, cũng như tất cả các thư viện lớn triển khai autodiff.

▼ Reverse mode autodiff sử dụng TensorFlow

```
import tensorflow as tf
```

```
x = tf.Variable(3.)
y = tf.Variable(4.)
```

```
with tf.GradientTape() as tape:
    f = x*x*y + y + 2
```

```
jacobians = tape.gradient(f, [x, y])
jacobians
```

```
[<tf.Tensor: shape=(), dtype=float32, numpy=24.0>,
 <tf.Tensor: shape=(), dtype=float32, numpy=10.0>]
```

Vì mọi thứ đều mang tính ký hiệu, chúng ta có thể tính toán đạo hàm cấp hai và xa hơn nữa:

```
x = tf.Variable(3.)
y = tf.Variable(4.)

with tf.GradientTape(persistent=True) as tape:
    f = x*x*y + y + 2
    df_dx, df_dy = tape.gradient(f, [x, y])

d2f_d2x, d2f_dydx = tape.gradient(df_dx, [x, y])
d2f_dxdy, d2f_d2y = tape.gradient(df_dy, [x, y])
del tape

hessians = [[d2f_d2x, d2f_dydx], [d2f_dxdy, d2f_d2y]]
hessians
```

```
WARNING:tensorflow:Calling GradientTape.gradient on a persistent tape inside its context is significantly less efficient than ca
[[<tf.Tensor: shape=(), dtype=float32, numpy=8.0>,
 <tf.Tensor: shape=(), dtype=float32, numpy=6.0>],
 [<tf.Tensor: shape=(), dtype=float32, numpy=6.0>, None]]
```

Lưu ý rằng khi chúng ta tính đạo hàm của một tensor theo một biến mà nó không phụ thuộc vào, thay vì trả về 0.0, hàm `gradient()` sẽ trả về `None`.

Và đó là tất cả! Hy vọng bạn thích notebook này.